Production Incident Report for NGINX Disk Exhaustion

AUTHOR : DZOANNGUYENTRAN@GMAIL.COM dEVOPS sre CLOUD ARCHITECT

Kepner Tregoe CERTIFIED TROUBLESHOOTER . https://kepner-tregoe.com/training/problem-solving-decision-making/

investigation and recovery | Illustrative case study

The supplied challenge establishes only an Ubuntu 24.04 VM with 64 GB of storage, an NGINX load balancer and a reported 99% storage alert. Everything else in this report—including the customer, names, architecture, ticket IDs, timeline, metrics, commands, logs, findings, remediation and confirmation—is a fictional worked example, not an actual incident or verified outcome.

1 Incident overview

Field	Illustrative record
Incident	INC-EXAMPLE-001	simulated production incident
Severity and owner	SEV-2 assumed; incident lead and on call DevOps engineer
Service	Example checkout API behind two NGINX load balancers
Alert	Root filesystem on lb-01 reached 99%
Customer impact	Some checkout requests routed through lb-01 failed or were slow; lb-02 remained healthy. Exact affected-user count is unknown.
Mitigation	Drained lb-01 from an existing upstream load balancer after verifying lb-02 capacity and configuration.
Root cause	Example verified cause: lb-01's custom error log was excluded from logrotate and grew rapidly when NGINX used an incorrect upstream port.
Recovery and closure	Traffic restored through lb-02; configuration and logging corrected; customer confirmation and closure are simulated below.

Summary: restore traffic first, recover disk headroom, test competing causes, repair the actual upstream configuration and log rotation, validate the customer journey, then document and prevent recurrence.

2 Open ticket and communicate

At 09:00 ICT, monitoring alerts on lb-01 root filesystem usage. The on call engineer opens the illustrative ticket, records time, component, impact, severity, owner and next customer update at 09:15. No actual ticket or message has been created.

Initial customer update, example: "We are investigating degraded checkout performance. Our team is assessing the load-balancing path and working to restore service. We will provide an update by 09:15 ICT. Reference: INC-EXAMPLE-001."

Do not claim a confirmed cause or restoration time until evidence supports it. Escalate using the organization's actual incident and communication policy.

3 Situation appraisal and dependency map

Illustrative architecture: customers → DNS → existing upstream traffic router → NGINX lb-01 or lb-02 → checkout API → database. The upstream router, second NGINX instance and checkout workload are assumptions for this case, not specified by the challenge.

Question	Illustrative finding
Is the service down or slow?	Some lb-01 requests return 502 or time out; checkout through lb-02 works.
Which users and services?	The checkout API and clients routed to lb-01; exact users and percentage unknown.
Where is the bottleneck?	The / filesystem on lb-01 is 99% full; backend connectivity also fails through its configured upstream.
Is a workaround available?	lb-02 has healthy targets, matching routes and certificates, and capacity for the full current traffic in this example.
What must be protected?	Customer traffic, incident evidence, NGINX availability and the ability to roll back configuration changes.

4 Incident timeline

All times below are invented for the example and in Asia/Ho Chi Minh City, UTC+07:00. They show a plausible sequence, not measured service levels.

Time	Action and result
09:00	Disk alert: lb-01 / at 99%; open ticket and assign on call owner.
09:03	Review dashboards: checkout latency and 502s elevated for traffic using lb-01.
09:05	Inform customer investigation is underway; next update scheduled.
09:08	Confirm architecture, route, TLS, backend access and spare capacity on lb-02.
09:10	Drain lb-01 via existing traffic router; validate checkout through lb-02.
09:13	Customer informed service appears restored; retain ticket and continue investigation.
09:16	Inspect df, du, effective NGINX config and rotation settings; collect evidence.
09:24	Identify error-log growth and omitted rotation path; test upstream connectivity.
09:31	Verify incorrect lb-01 upstream port and confirm peer uses correct port.
09:38	Preserve needed evidence and remove approved inactive archives; regain headroom.
09:48	Correct and test upstream and log rotation configuration under approved change.
10:00	Validate upstream, NGINX, rotated logs, free space and synthetic checkout.
10:10	Return a small share of traffic to lb-01 and monitor.

Time	Action and result
10:20	Confirm metrics stable; customer reports checkout works in this example.
10:30	Resolve and close ticket under policy; publish incident summary and KB follow-ups.

5 Traffic mitigation and immediate recovery

Before draining lb-01, verify lb-02 health, capacity, server names, route mappings, certificates, authorization and reachability to the checkout API. Use the organization's existing load-balancer console or automation; no provider-specific command is assumed. Drain active connections when possible and consider sticky sessions and DNS TTLs.

The incident lead chooses rerouting because it is reversible and restores the customer journey faster than repairing a nearly full active VM. If the second instance is unavailable or cannot support the workload, use an approved replacement or recover safe disk headroom while monitoring traffic. Record the chosen action, time, checks and rollback.

6 KT problem analysis IS and IS NOT

Deviation statement: lb-01's root filesystem reached 99% and requests sent through it returned errors; the reason for the disk growth and connection errors was initially unknown.

Dimension	IS example observations	IS NOT comparison and distinction
What	Root filesystem bytes are 99% used; NGINX emits upstream connection errors.	Inodes are not exhausted. A disk alert alone does not establish customer impact.
Where	lb-01 root filesystem; custom error log is large.	lb-02 has sufficient disk and routes checkout successfully. Peers are compared after checking configuration and load.
When	Disk growth and NGINX errors began following a configuration deployment.	Baseline was normal before the change in this example. Timing is a clue, not proof.
Extent	One VM affected; error log accounts for most local used space.	No similar alert on lb-02; the other mounts remain healthy.

7 Evidence collection and diagnostic commands

The commands below are examples. NGINX configuration output and logs may contain sensitive data; record only sanitized evidence in the ticket. First identify the actual mount point; du -x stays on its filesystem.

```
df -hT
df -i
findmnt -T /var/log/nginx
lsblk -f
sudo systemctl status nginx --no-pager
sudo ss -lntp
```

```
sudo du -xhd1 / 2>/dev/null | sort -h
sudo du -xhd1 /var/log 2>/dev/null | sort -h
sudo ls -lh /var/log/nginx/
sudo lsof +L1
sudo journalctl --disk-usage
```

```
sudo nginx -t
sudo nginx -T 2>/dev/null | grep -E "access_log|error_log|proxy_pass|upstream"
sudo cat /etc/logrotate.d/nginx
sudo systemctl status logrotate.timer logrotate.service --no-pager
```

Example output, simplified: df shows / at 99%; du shows /var/log is dominant; the custom error log is approximately 46 GiB; inode usage is 12%; journal and cache are small. Values are illustrative and rounded, and no account of real data is implied.

Evidence must establish whether /var/log is on the same mount, whether large files are linked or deleted but open, what NGINX actually uses and whether log rotation covers each path. Collect application metrics, recent changes and comparable configuration from lb-02.

8 Hypotheses and tests

Possible cause	Expected evidence and test	Illustrative result
Custom error log is not rotated	The log dominates usage; effective config names it; inspect all included rotation rules for its path.	Supported: the configured custom path is absent from effective rotation rules.
Upstream fault generates log volume	Repeated errors match a reproducible connection failure and an observable configuration difference.	Supported: lb-01 points to port 8081; approved checkout endpoint listens on 8080. Peer uses 8080.
Deleted file remains open	df exceeds visible du; lsof +L1 shows large deleted files on the affected mount.	Not supported: no relevant large deleted-open file in the example.
Journal, cache or inodes	Corresponding usage dominates or df -i is exhausted.	Not supported: small journal/cache and available inodes.
Unexpected activity or excess external traffic	Unknown files/processes or unusual source-volume patterns.	Not indicated by examined evidence; security review remains appropriate if signs arise.

9 Confirmed root cause and contributing factors

In this constructed case, the engineer confirms the custom log path is not covered by logrotate. A comparison and targeted connectivity test show lb-01 was configured to proxy checkout traffic to port 8081 even though the approved service listens on 8080. The incorrect port produces connection-refused errors; the unrotated log accumulates those errors until the filesystem fills.

The upstream configuration error caused customer request failures and error-log volume. The missing rotation policy allowed disk consumption to accumulate. Neither the alert nor time correlation alone proves this chain; each link is checked through effective configuration, a relevant connection test, log pattern and disk attribution.

Example connection test after checking the actual hostname and port: curl --connect-timeout 2 --max-time 5 http://UPSTREAM_HOST:8080/health. The host and endpoint are placeholders, and an HTTP response alone does not validate all NGINX routing or TLS behavior.

10 Repair and permanent correction

Immediate headroom

Preserve required logs and security evidence first. In this example an inactive 8 GiB rotated archive is confirmed eligible for deletion under the retention policy. After authorized deletion, disk usage falls from about 99% to about 86%, providing temporary headroom. The archive name and result are illustrative; do not delete active or unidentified files.

Example only, after identifying and approving the exact archived file: sudo rm -i -- /var/log/nginx/APPROVED_OLD_ARCHIVE.gz. An actually existing approved filename must replace the placeholder. Verify df -hT afterward. Never assume normal log rotation alone immediately frees space.

Permanent fixes

- Correct the checkout upstream port in the approved configuration source. Validate that the expected endpoint is reachable from lb-01, run sudo nginx -t, then perform an approved reload. Confirm no undesired routes or TLS settings changed.
- Add the actual custom error log to the approved rotation policy with correct permissions, retention and NGINX log reopening. Test the rules in a safe environment and review the postrotate action.

```
sudo nginx -t
sudo logrotate -d /etc/logrotate.conf
sudo logrotate /etc/logrotate.conf
```

logrotate -d is a dry run. Normal rotation applies only when due and may remove older archives according to policy; check requirements first. A forced rotation is not a default remedy and must be separately assessed for retention, available disk space and reopen behavior.

- Investigate why the bad port was introduced. Add automated config review and smoke tests against the expected upstream. Export logs centrally, alert on time to full and logrotate failures, and test failover capacity.

Decision and risk review

The chosen repair is reversible: retain the prior approved configuration and record the change ID. Before reload, verify syntax and upstream connectivity; after reload, check service health and customer requests. If error rate rises, drain the target and restore the previous known-good configuration. Do not use a restart as the first reaction when it could worsen an outage.

11 Recovery validation

Validation	Illustrative result and evidence
Disk / inodes	Root usage ~86% after approved archive cleanup and stable; inode usage remains healthy.
NGINX	nginx -t passes; service stays active; current log is writable.
Upstream	Effective configuration uses approved port; connectivity and application health test succeed.
Logging	Approved rotation configuration covers the custom file; dry-run output and reopen procedure verified; full cycle tested in safe environment.
Customer path	Synthetic checkout succeeds end to end through lb-01 and lb-02.
Traffic and metrics	Traffic returned gradually; error rate and latency return to the normal baseline for an agreed observation period.
Customer	Example customer confirms checkout works at 10:20 ICT. This is fictional and no actual customer was contacted.

```
df -hT
df -i
sudo lsof +L1
sudo nginx -t
sudo systemctl status nginx --no-pager
```

12 Customer updates and closure

Illustrative mitigation update at 09:13: “We routed checkout traffic to a healthy load balancer. Our checks show service has improved; we are investigating the original server and will update you by 09:45 ICT.”

Illustrative resolution request at 10:10: “We corrected the load-balancer configuration and recovered disk capacity. Our end-to-end tests are passing. Please confirm checkout works for you; we are monitoring the service.”

The example customer confirms at 10:20, and the incident is closed at 10:30 in accordance with the assumed policy. Actual closure must use real confirmation and policy. If a customer does not respond, record pending confirmation and follow the organization's escalation/closure rules. If permanent corrections are incomplete, link a problem or change record with an owner and due date; do not mark them done.

13 Follow up and knowledge base entry

Action	Illustrative owner and status
Add custom log path to managed rotation and verify configuration	Platform engineer — complete in example; retain change evidence.
Correct upstream config and add automated deployment smoke test	Application and platform owners — port corrected; automated test tracked as follow up.
Add disk, inode, time-to-full, error-rate and rotation-failure alerts	Observability owner — open; set due date according to policy.
Test load-balancer failover and surviving capacity	SRE owner — open; schedule a game day.
Publish sanitized incident article with evidence and rollback	Incident owner — link incident, change and KB; set review date.

KB entry must capture symptoms and impacted traffic, architecture and failover, the KT IS / IS NOT table, commands and sanitized output, rejected hypotheses, verified cause, exact corrective change, before/after validation, customer confirmation, prevention owners and pending work. Do not place credentials, customer data or unredacted sensitive logs in the KB.

14 References

NGINX control and log reopening: https://nginx.org/en/docs/control.html

Ubuntu 24.04 logrotate: https://manpages.ubuntu.com/manpages/noble/man8/logrotate.8.html

Ubuntu 24.04 df: https://manpages.ubuntu.com/manpages/noble/man1/df.1.html

All findings, actions, times, outcomes and customer communications in this report are illustrative; validate the actual environment and follow local operational policy.